Kingdom of Saudi Arabia

Ministry of Education

King Faisal University

College of Computer Sciences & Information Technology

Smart Plants Irrigation System

Embedded Systems Project Report

Prepared by:

Abdullah Ali Alkhulaifi - 223002824

Bashar Ibrahim Farhat - 223039656

Ayed Ahmed Almutairi - 223041875

Abdulelah Adel Alhinity - 223023047

Supervised by:

Dr. Ramasamy

Date: April 2026

Abstract

This project implements a smart plant irrigation system using two ESP32 microcontrollers. One ESP32 works as a soil sensing node and measures soil moisture. The second ESP32 works as the main controller: it reads the water tank level, controls a relay connected to a 5 V water pump, receives soil readings over Wi-Fi, and hosts a browser-based dashboard. The pump is powered from a 3.8 V battery through a step-up converter that raises the voltage to the required 5 V level. The system supports automatic irrigation, manual control, a timed override mode, and a water-level safety check to reduce the chance of running the pump when the tank is low.

1. Introduction

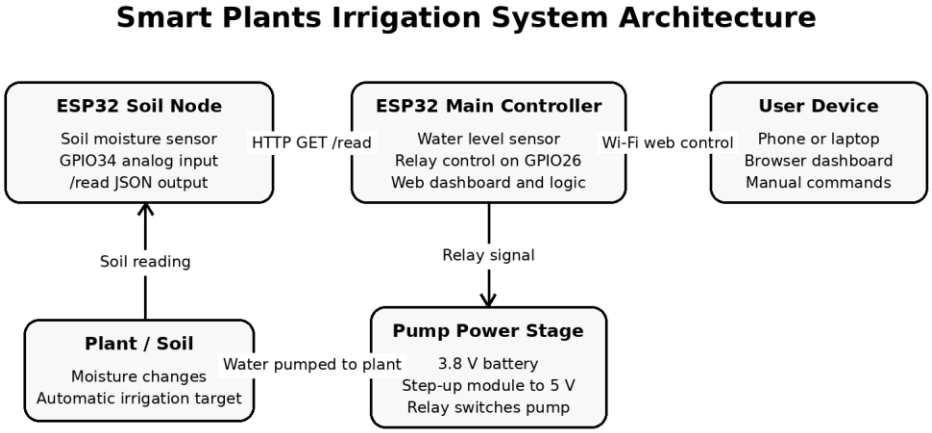
Small plants are often damaged by irregular watering: either the soil becomes too dry, or the plant receives water when it is not needed. The purpose of this project is to automate the basic watering decision using moisture and water-level feedback. Instead of using one large controller for everything, the design separates the soil sensor and the pump controller into two ESP32 units. This makes the design easier to place physically: the soil sensor can remain near the plant, while the pump controller can remain near the water tank, relay, battery, and pump.

2. Project Objectives

- Measure soil moisture and classify the soil as wet or dry.
- Measure the water tank level and convert the raw ADC reading to a percentage.
- Automatically turn the pump on when the soil is dry and stop it when the soil is wet.
- Prevent normal pump operation when the tank water level is below a safe threshold.
- Provide a simple web dashboard for status monitoring and manual control.
- Use battery power for the pump by stepping a 3.8 V battery up to 5 V.

3. System Overview

The system is divided into two ESP32 subsystems connected through the same Wi-Fi network. The soil node exposes one HTTP endpoint that returns JSON. The main controller periodically requests this JSON reading, combines it with its own water-level measurement, and decides whether the relay should energize the pump.



The two ESP32 boards communicate over the same Wi-Fi network. The main controller decides when the pump can run.

Figure 1. Overall system architecture.

Subsystem	Main role	Important code features
Soil sensor ESP32	Reads soil moisture near the plant.	Analog input on GPIO34, averaging, dry/wet calibration, /read JSON endpoint.
Main controller ESP32	Controls the pump and hosts the dashboard.	Water sensor on GPIO34, relay on GPIO26, HTTPClient request to soil node, dashboard routes.
Pump power stage	Supplies and switches the pump.	3.8 V battery, step-up converter to 5 V, relay switching.
User interface	Allows monitoring and control.	HTML/CSS/JavaScript dashboard served directly from the main ESP32.

4. Hardware Components

Component	Purpose in the project
ESP32 board 1	Soil moisture node. It reads the soil sensor and publishes readings through Wi-Fi.
ESP32 board 2	Main controller. It reads water level, controls the relay, and hosts the web dashboard.
Soil moisture sensor	Provides an analog value related to the amount of water in the soil.
Water level sensor	Provides an analog value used to estimate the water tank percentage.
Relay module	Electrically switches the pump on and off from the ESP32 control signal.
5 V water pump	Moves water from the tank to the plant.
3.8 V battery	Portable power source for the pump system.
Step-up converter	Raises the 3.8 V battery voltage to approximately 5 V for the pump.
Tubing and water tank	Directs water from the tank to the plant.

5. Software Design

The software is written using the Arduino framework for ESP32. It uses WiFi and WebServer on both boards. The main controller also uses HTTPClient to request the soil node reading. This design avoids needing wires between the two ESP32 boards.

5.1 Soil Moisture Node

The soil node is the simpler ESP32. It reads analog values from the soil sensor on GPIO34, averages multiple samples to reduce noise, maps the raw value into a moisture percentage, and sends

the result as JSON. In the submitted code, SOIL_DRY_RAW is 3000, SOIL_WET_RAW is 1200, and the dry threshold is 35 percent. Because the sensor gives a higher raw value when dry and a lower raw value when wet, the mapping is reversed: dry raw maps near 0 percent and wet raw maps near 100 percent.

Function / variable	Role
readSoilAverage()	Takes 15 analog samples and returns their average.
calculateSoilPercent()	Maps the raw soil value to a 0-100 moisture percentage.
getIsDry()	Returns true when moisture is at or below the dry threshold.
/read	HTTP endpoint returning raw, percent, and isDry as JSON.

5.2 Main Controller Node

The main controller handles the decisions and outputs. It reads the water level sensor on GPIO34 and controls a relay on GPIO26. The relay is configured as active-low, so a LOW output energizes the relay and a HIGH output turns it off. The controller hosts a dashboard page and a /status endpoint, which returns the latest water, soil, pump, mode, timer, and command information as JSON.

Feature	Implementation in the main controller
Water reading	31 ADC samples are collected, sorted, and the middle samples are averaged to reduce noise and reject outliers.
Water calibration	Raw values are mapped from WATER_EMPTY_RAW = 1343 to WATER_FULL_RAW = 1633.
Hysteresis	Small changes within 3 percent are ignored to reduce display and control flicker.
Soil communication	The controller sends HTTP GET requests to http://SENSOR_IP/read.
Pump control	pumpON(), pumpOFF(), and forcePumpON() update both relay state and system status.
Dashboard	A responsive HTML page shows water, soil, pump, mode, timer, and last command.

6. Control Logic

The control logic is based on mode selection. The project includes automatic mode, manual mode, and override mode. The main loop updates the water sensor every second and requests the soil sensor every five seconds. This gives fast water protection while avoiding unnecessary network requests to the soil node.

6.1 Automatic Mode

1. The main controller requests the latest soil moisture value from the soil ESP32.
2. If the reading is valid, the controller stores raw value, percentage, and dry/wet status.
3. If the soil is dry and the pump is stopped, pumpON() is called.

4. Before the pump turns on, the controller checks that power is enabled and the water level is not below the minimum threshold.
5. If the soil is wet and the pump is running, pumpOFF() is called.

6.2 Manual Mode

Manual mode allows the user to control the pump from the dashboard. The Pump ON button is only shown when automatic mode is disabled. Even in manual mode, pumpON() still checks the water level before allowing normal pump operation.

6.3 Override Mode

Override mode runs the pump for a user-selected time. The timer value is limited from 1 to 3600 seconds. In the current code, override mode uses forcePumpON(), which bypasses the normal water-level check. This makes override useful for testing, but it should be treated carefully. A safer final design would also block override mode when the tank is below the minimum water level.

6.4 Safety Conditions

- The pump is normally blocked if waterPercent is below MIN_WATER_PERCENT_TO_RUN, which is 20 percent in the code.
- The relay state is synchronized using relayIsOn() so the software status matches the actual relay output.
- If water becomes too low while the pump is running outside override mode, the controller turns the pump off.
- If Wi-Fi is disconnected, the controller cannot fetch new soil readings, so automatic decisions depend on the last valid reading.

7. Calibration and Sensor Processing

Calibration is important because analog sensors rarely use the full ESP32 ADC range. The system does not assume that 0 means empty or 4095 means full. Instead, it uses measured raw values for the actual sensors and environment.

Sensor	Calibration method	Reason
Soil moisture	SOIL_DRY_RAW = 3000 and SOIL_WET_RAW = 1200 are mapped to 0-100 percent.	The soil sensor has opposite behavior: dry gives a higher raw value and wet gives a lower raw value.
Water level	WATER_EMPTY_RAW = 1343 and WATER_FULL_RAW = 1633 are mapped to 0-100 percent.	The real measured water sensor range is narrow, so using raw / 4095 would give inaccurate percentages.
Noise reduction	Averaging and trimmed averaging are used.	ADC readings can fluctuate because of sensor noise, wiring, and power variation.
Hysteresis	Changes within 3 percent are ignored.	This prevents rapid switching around threshold values.

8. Web Dashboard and Communication

The main ESP32 serves a web dashboard that can be opened from a phone or laptop connected to the same Wi-Fi network. The dashboard updates itself by requesting /status every two seconds. It shows water raw value, water percentage, water status, soil raw value, soil moisture percentage, soil status, pump state, operation mode, override status, timer value, power state, and the last command. The interface includes Arabic and English labels, making it easier to demonstrate to different users.

9. Power and Relay Design

The water pump requires 5 V, while the available battery is approximately 3.8 V. For this reason, a step-up converter is used between the battery and the pump supply path. The ESP32 does not drive the pump directly; it only drives the relay input. The relay then switches the pump circuit. This separation is necessary because the pump current is higher than what an ESP32 GPIO pin can safely provide.

- The ESP32 GPIO controls the relay signal, not the pump motor directly.
- The relay module isolates the low-power control side from the higher-current pump side.
- The step-up converter should be rated for the pump startup current, not only the normal running current.
- Common ground and stable wiring are important to avoid relay resets, wrong ADC readings, or ESP32 restarts.

10. Testing Using the Web Interface

Test	Expected result
Soil node test	JSON appears with raw, percent, and isDry.
Dry soil test	Moisture percentage decreases and isDry becomes true.
Wet soil test	Moisture percentage increases and isDry becomes false.
Water sensor test	Water percentage changes according to calibration.
Automatic irrigation test	Pump turns on automatically.
Water safety test	Pump is blocked or stopped.
Manual mode test	Pump responds through the relay when safety conditions allow it.
Override timer test	Pump runs for the selected duration and then stops.

11. Simulation Scenarios for Different Plant Seeds

To evaluate the practical capability of the system, the irrigation logic can be simulated with different plant or seed profiles. This section does not claim exact agricultural watering values because the final requirement depends on pot size, soil type, drainage, room temperature, and the actual pump flow rate. Instead, the table translates common seedling-care behavior into usable prototype profiles: keep the soil moist enough for germination, avoid soggy soil, and water using short pump pulses followed by a new sensor reading.

The moisture values below are interpreted using the project soil sensor scale, where 0 percent means very dry and 100 percent means very wet based on the submitted calibration. The current soil ESP32 code uses one global dry threshold of 35 percent. For a stronger final version, the selected plant profile could change this threshold and the override timer from the dashboard.

Plant / seed type	Moisture target and dry trigger	Suggested pump pulse	Simulation behavior
Lettuce	Target: 55-70 percent; dry below 45 percent	3-5 seconds	Light, frequent watering. The pump should run briefly because lettuce seeds and seedlings need consistent moisture but can suffer if the soil stays soaked.

Plant / seed type	Moisture target and dry trigger	Suggested pump pulse	Simulation behavior
Spinach	Target: 55-70 percent; dry below 45 percent	3-5 seconds	Similar to lettuce. The system should avoid long pumping and instead check moisture again after a short delay.
Basil	Target: 50-65 percent; dry below 40 percent	4-6 seconds	Moderate moisture profile. The pump runs only when the soil drops below the dry trigger, then stops after a short pulse.
Tomato	Target: 60-75 percent; dry below 50 percent	6-8 seconds	Tomato seedlings usually need deeper moisture than leafy greens. The controller can use a higher target and a longer pulse.
Pepper	Target: 50-65 percent; dry below 40 percent	5-7 seconds	Pepper seedlings should not be overwatered. The system should keep moisture moderate and allow slight drying before the next cycle.
Cucumber	Target: 60-75 percent; dry below 50 percent	6-10 seconds	A higher-water profile. The pump may run longer, but the tank safety condition should still block watering if water level is below 20 percent.

Example simulated operation: if the selected profile is tomato and the soil moisture reading is 42 percent, the system classifies the soil as below the tomato dry trigger. If the water tank is at or above 20 percent and the system is in automatic mode, the pump can run for about 6-8 seconds. After the pulse, the system waits for the soil reading to update, then either stops or repeats with another short pulse if the soil is still below the target range.

Capability	How the current system supports it	Required improvement for plant profiles
Different plant moisture targets	The soil node already calculates moisture percentage.	Replace the fixed 35 percent dry threshold with selectable profile thresholds.
Different pump durations	Override mode already supports a timer from 1 to 3600 seconds.	Use the same timer concept automatically based on the selected plant profile.
Water safety	The main controller blocks normal pumping below 20 percent tank level.	Keep this protection active in profile-based watering and also apply it to override mode.
Closed-loop correction	The main ESP32 fetches soil readings every 5 seconds.	After each pulse, wait for moisture to stabilize before deciding on another pulse.

For a real calibration, pump duration should be measured instead of guessed. The practical method is to run the pump for 10 seconds into a measuring cup, record the delivered water volume, then calculate the pump flow rate in milliliters per second. After that, each plant profile can convert a desired water volume into a timer value. Without this measurement, the pulse durations in the table should be treated as demonstration values only.

All these results were taken from the Excel file with the simulated results. For more simulation results, refer to the Excel file in the references

12. Results and Discussion

The project demonstrates a working closed-loop irrigation concept. The soil sensor provides the plant condition, the water sensor provides tank safety information, and the relay controls the physical pump. Separating the system into two ESP32 boards is useful because it reduces wiring between the soil area and the pump area. The browser dashboard also makes the system easier to demonstrate because no mobile application is required.

The main engineering issue in this project is sensor calibration. Both the soil sensor and the water sensor depend heavily on real measured raw values. For the water sensor, the calibrated range in the code is 1343 to 1633, so even a small raw-value change can create a large percentage change. This explains why stable wiring, repeated sampling, and recalibration are important. The code addresses this partly by using multiple samples, trimmed averaging, and percentage hysteresis.

13. Limitations

- The ESP32 boards use a fixed SENSOR_IP value. If the soil node IP changes, the main controller will not find it until the code is updated or the network assigns the same address again.
- Wi-Fi credentials are hardcoded in the sketches. This is acceptable for a lab prototype but not ideal for a final product.
- The system has no login or password on the dashboard, so anyone on the same network could control it.
- The JSON parsing in the main controller is manual string parsing. A JSON library would be more reliable if the project grows.
- Override mode currently bypasses the normal water-level protection, which can be risky for the pump if the tank is empty.
- The water level sensor calibration values are very close together, so physical placement and electrical noise can strongly affect the displayed percentage.

14. Future Improvements

- Add a safer override mode that still refuses to run the pump when the water level is too low.
- Reserve static IP addresses on the router or use mDNS names instead of hardcoding SENSOR_IP.

- Add a Wi-Fi setup page so SSID and password do not need to be edited in the source code.
- Use ArduinoJson for cleaner and safer JSON parsing.
- Add authentication to the dashboard before allowing pump commands.
- Store calibration values in non-volatile memory so they can be changed from the dashboard.
- Add a real-time clock or logging feature to record watering history.
- Improve the power circuit by checking pump current, step-up converter rating, and battery protection.

15. Conclusion

The Smart Plants Irrigation project is a practical embedded systems application that combines sensing, networking, actuation, and user interaction. The two ESP32 boards communicate over Wi-Fi, the soil node provides moisture information, and the main controller makes pump decisions using both soil condition and water-level safety. The system is suitable for a course demonstration because it includes analog sensor reading, calibration, HTTP communication, relay control, a web interface, and power conversion for the pump. With stronger safety handling, better calibration storage, and basic access control, the same prototype could be developed into a more reliable smart irrigation product.